RRT* Motion Planning with Control Barrier Function Safety Derived from Poisson Fields

Maciej Różewicz

***Abstract*— This paper proposes a novel framework for safe motion planning that integrates Control Barrier Function (CBF) safety constraints with the Rapidly-Exploring Random Tree Star (RRT*) algorithm. The CBF is derived directly from the solution of a Poisson equation defined over the free workspace, referred to as a *Poisson Safety Function (PSF)*. The PSF encodes obstacle boundaries and safety gradients in a smooth, differentiable field that naturally satisfies Laplace-type spatial regularity. The proposed method, termed *Safe-CBF-RRT**, uses the PSF and its spatial derivatives to enforce pCBF-based safety conditions along sampled edges during tree expansion. Unlike pure safety-filter approaches such as FaS-Track, our method actively generates an inherently safe path while maintaining computational tractability. Simulation results demonstrate that the method enhances safety and convergence properties compared to standard RRT*.**

I. Introduction

In modern robotics applications, ensuring safe navigation in complex environments is one of the major challenges. This is particularly critical in scenarios involving human-robot interaction, autonomous vehicles, and unstructured terrains. Traditional motion planning algorithms often focus on finding feasible paths without formal guarantees of safety. To achieve such provable safety, Control Barrier Functions (CBFs) have emerged as a powerful tool for enforcing safety constraints in control systems in last years [1].

In parallel, methods like **FaSTrack** [4] have provided formal guarantees for *safe trajectory tracking* by defining a Safe-to-Track Set ($\mathcal{S}$), leveraging techniques from optimal control and differential games. However, such methods are primarily safety filters for the *execution* phase and *do not inherently provide an efficient planner* to *generate* the safe reference trajectory itself, often suffering from high computational cost in high-dimensional state spaces.

Sampling-based motion planners such as RRT or RRT* are widely used for complex robotic systems due to their scalability and ability to handle nonconvex constraints [7], [6]. However, they rely primarily on geometric collision checking, offering no formal guarantees of safety. For this reason, integrating CBFs into sampling-based planners seems to be a promising direction to enhance safety while retaining the benefits of sampling-based methods.

A critical challenge in using CBFs for planning is the construction of a suitable safety function $h(x)$, which must be continuously differentiable and satisfy certain regularity conditions. Analytical definitions of $h(x)$ exist for simple geometries, and lately there have been attempts to get CBF as solution of Poisson equation [3], [2]. This approach will be used in this paper to generate a Poisson Safety Function (PSF) that encodes obstacle boundaries and provides smooth safety gradients. Such definition of $h(x)$ is particularly advantageous for complex environments where analytical forms are not feasible. Moreover, it seems to be naturally compatible with the RRT or RRT* framework.

Email: rozewicz.maciej@gmail.com

Contributions: In this the *Safe-CBF-RRT** framework, the following contributions are made:

- Integration of the PSF into a modified RRT* algorithm.
- A computationally efficient alternative to tracking-based safety methods like FaSTrack for generating inherently safe reference trajectories.
- Empirical results showing improved safety margins and convergence.

II. Related Work

Sampling-based methods such as RRT and RRT* [6] have become standard in high-dimensional motion planning. Extensions incorporating optimality, dynamics, and risk awareness exist but often lack formal safety guarantees.

Potential field and PDE-based navigation [**?**] have long been studied for path planning. Harmonic potential fields, governed by Laplace's equation, ensure smoothness and avoid local minima, yet they do not directly provide a safety margin measurable in the CBF framework.

Control Barrier Functions [**?**], [**?**] ensure set invariance in control systems but require a known differentiable safety function. Existing methods often define $h(x)$ as analytic distances to obstacles, which may be nondifferentiable in complex maps. Our approach bridges this gap by generating $h(x)$ through a Poisson PDE that respects boundary geometry.

A. Integration of CBFs in Sampling-based Planning

The integration of Control Barrier Functions with sampling-based planners often falls into two main categories: those focusing on *control synthesis* and those focusing on *path filtering*.

1) Dynamic and Robust CBF-RRT* (Control Synthesis): More advanced methods directly incorporate the system's dynamics into the planning process. For example, the Robust CBF-RRT* approach [9] uses CBF conditions, often solved via Quadratic Programming (QP), at each time step along a sampled edge to find a dynamically safe control input u. This approach guarantees safety for kinodynamic systems and can be extended to Robust CBF (RCBF) to handle bounded uncertainties [9]. Similarly, methods like LQR-CBF-RRT*

aim for optimality by integrating the Linear Quadratic Regulator (LQR) with CBF constraints, sometimes simplifying the process by avoiding the continuous QP solving [8]. While powerful, these methods incur a high online computational cost due to the repeated synthesis of safe control (e.g., solving QPs or LQR) during tree expansion.

2) Geometric CBF-RRT⋆ (Path Filtering): In contrast, our method, *Safe-CBF-RRT⋆*, uses a computationally lighter approach. It employs a simplified, discrete CBF condition applied to the path segment geometry rather than synthesizing a dynamic control trajectory. This shifts the focus from finding an optimally safe control to generating an inherently safe reference path with minimal online computational overhead.

Contrast with Tracking-Based Safety Methods (FaSTrack): Another influential paradigm for provable safety is the *FaSTrack* framework [4], which uses level-set methods and Hamilton-Jacobi-Isaacs (HJI) equations to compute the aforementioned Safe-to-Track Set ($\mathcal{S}$). While FaSTrack offers strong, worst-case guarantees against bounded disturbances and execution error, **it decouples planning from safety assurance**. The path must first be planned (e.g., via RRT⋆), and then the safe-to-track set is checked for feasibility, often leading to computationally demanding solutions for high-dimensional systems. In contrast, *Safe-CBF-RRT⋆* integrates the safety constraint derived from the smooth Poisson field directly into the sampling and rewiring steps. This allows the planner to efficiently *generate inherently safe paths*, avoiding the need for expensive high-dimensional HJI solutions characteristic of FaSTrack-type approaches.

[3]

III. Preliminaries

A. RRT⋆ Algorithm

The RRT⋆ algorithm is an incremental, sampling-based motion planner that asymptotically converges to the optimal feasible path. At each iteration, a random sample x_{rand} is drawn from the state space, and the tree is extended toward this sample from its nearest node x_{near}. Unlike RRT, RRT⋆ performs two additional operations that guarantee asymptotic optimality. First, it selects a parent for the new node x_{new} by choosing, among all nodes within a ball of radius r_n, the one that minimizes the cost-to-come plus the local connection cost. Second, it "rewires" the tree by checking whether any neighbor can reduce its cost by adopting x_{new} as a new parent. As the number of samples n grows, the radius $r_n = \gamma(\frac{\log n}{n})^{\frac{1}{d}}$ shrinks, ensuring both probabilistic completeness and convergence to the optimal path cost. This makes RRT⋆ suitable for high-dimensional motion planning problems where exact optimal planning is computationally intractable [5], [6].

Algorithm is summarized in Algorithm 1.

Algorithm 1 Classical RRT⋆ algorithm

Require: State space $\mathcal{X}$, free space $\mathcal{X}_{\text{free}} \subseteq \mathcal{X}$, initial state x_{init}, number of iterations N

$T \leftarrow (\{x_{\text{init}}\}, \emptyset)$
for $n = 1$ **to** N **do**
  $x_{\text{rand}} \sim \text{Unif}(\mathcal{X}_{\text{free}})$
  $x_{\text{near}} \leftarrow \text{Nearest}(T, x_{\text{rand}})$
  $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{near}}, x_{\text{rand}})$
  if $\text{CollisionFree}(x_{\text{near}}, x_{\text{new}})$ **then**
    $\mathcal{N} \leftarrow \text{Neighbors}(T, x_{\text{new}}, r_n)$
    $x_{\text{parent}} \leftarrow \arg\min_{x \in \mathcal{N}} (c(x) + \text{Cost}(x, x_{\text{new}}))$
    $T.\text{AddNode}(x_{\text{new}})$
    $T.\text{AddEdge}(x_{\text{parent}}, x_{\text{new}})$
    for all $x \in \mathcal{N}$ **do**
      if $c(x_{\text{new}}) + \text{Cost}(x_{\text{new}}, x) < c(x)$ **then**
        $T.\text{Rewire}(x, x_{\text{new}})$
      end if
    end for
  end if
end for

return T

B. Control Barrier Functions

Control Barrier Functions (CBFs) provide a formalism for ensuring safety in control systems by enforcing forward invariance of a safe set. Given a dynamical system

$$\dot{x} = f(x) + g(x)u, \tag{1}$$

where $x \in \mathbb{R}^n$ is the state, $u \in \mathbb{R}^m$ is the control input, and f, g are locally Lipschitz continuous functions, a set

$$\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\} \tag{2}$$

is defined as *safe* if there exists a continuously differentiable function $h : \mathbb{R}^n \to \mathbb{R}$ such that for all $x \in \mathcal{C}$,

$$\sup_{u \in U} [L_f h(x) + L_g h(x) u] \geq -\kappa h(x), \tag{3}$$

where $L_f h$ and $L_g h$ are the Lie derivatives of h along f and g, respectively, and $\kappa > 0$ is a tuning parameter - the smaller κ is the more conservative is condition. This condition ensures that the system's trajectories remain within the safe set $\mathcal{C}$ for all future times.

C. Poisson Safety Function Generation

Let $\Omega \subset \mathbb{R}^2$ denote the workspace with boundary $\partial\Omega$ and obstacles $\mathcal{O}_i$. We define a vector potential field $\mathbf{u} = [u_x, u_y]^T$ solving

$$\left\{ \begin{array}{ll} \nabla^2 \mathbf{u} = 0 & \text{in } \Omega \\ \mathbf{u} = c_i(\mathbf{x} - \mathbf{c}_i) & \text{on } \partial\mathcal{O}_i \end{array} \right\}. \tag{4}$$

A scalar field $h(x, y)$, termed the *Poisson Safety Function (PSF)*, is then obtained by solving

$$\left\{ \begin{array}{ll} \nabla^2 h = -\|\nabla \mathbf{u}\| & \text{in } \Omega \\ h|_{\partial\Omega} = 0 & \text{on } \partial\mathcal{O}_i \end{array} \right\}, \tag{5}$$

ensuring $h > 0$ in free space. Gradients $\nabla h = [\partial h/\partial x, \partial h/\partial y]$ are computed using finite differences.

IV. Safe CBF-RRT⋆ Algorithm

A. Ensuring Safety with Poisson CBFs

Safe CBF-RRT⋆ integrates the PSF-based CBF constraint into the RRT⋆ framework to ensure safety during tree expansion. The PSF $h(x)$ and its derivatives are used in following ways:

- $h(x)$, $\frac{\partial h(x)}{\partial x}$, $\frac{\partial h(x)}{\partial y}$ are used to fast check if sampled edge is CBF-safe - assume simple constant velocity model along edge,
- $h(x)$ is used to weight path costs - the higher the average $h(x)$ along the edge, the safer the edge is considered,
- if simple traversing along the edge is safe then $h(x)$, $\frac{\partial h(x)}{\partial x}$, $\frac{\partial h(x)}{\partial y}$ are used to check if control inputs can be found to keep the system safe with smooth trajectory.

B. Trajectory cost with safety weighting

Safe CBF-RRT⋆ extends RRT⋆ by filtering edges through CBF validation and weighting path costs by safety:

$$J(a,b) = cL + (1-c)\frac{1}{\bar{h}}, \tag{6}$$

where $L = \|b-a\|$, $\bar{h}$ is mean h along the edge, and $c \in [0,1]$ balances length and safety.

C. Summary of Safe-CBF-RRT⋆

The planner iteratively samples, extends, and rewires as in RRT⋆, rejecting edges that violate the CBF condition. A pseudocode outline is shown in Algorithm 2.

Algorithm 2 Safe-CBF-RRT⋆

```
Initialize tree with x_start
for k = 1 : N_iter do
    Sample x_rand
    x_near ← Nearest(x_rand)
    x_new ← Steer(x_near, x_rand)
    if CollisionFree(x_near, x_new) and CBFSafe(x_near, x_new) then
        AddNode(x_new); RewireNeighborhood(x_new)
    end if
    if x_new near x_goal then
        Connect and return path
    end if
end for
```

V. Results

This section presents simulation results comparing Safe-CBF-RRT⋆ against standard RRT⋆ in a 2D environment with randomly located obstacles. It provides diversity of paths, safety margins, and path lengths.

A. Simulation Setup

The workspace was a square randomly located obstacles. The grid resolution was 100×100. Parameters used were stepSize $= 2$, $\kappa = 20$, and $N_{iter} = 2000$.

There were three scenarios with varying obstacle densities and configurations:

- **Scenario I:** Moderately spaced obstacles with wide corridors.
- **Scenario II:** Tightly packed obstacles with narrow passages.
- **Scenario III:** Sparse obstacles with large open areas.

Each scenario was run 100 times for both algorithms - it means with $c = 1$ (length optimized only) and $c = 0$ (safety optimized only), recording path length, average safety metric $\bar{h}$, number of iterations, and rejection ratio to show how many potntially unsafe segmets are rejected from tree.

Additionally for both algorithm calibrations corresponding iterations were run with same random seed to show how both algorithms behave in exactly same conditions.

B. Performance

1) Scenario I: Scenario I represents an environment with moderately spaced obstacles and relatively wide free-space corridors. The PSF field (Fig. 1) exhibits smooth gradients and well-defined safety basins around obstacles, making it a representative baseline configuration.

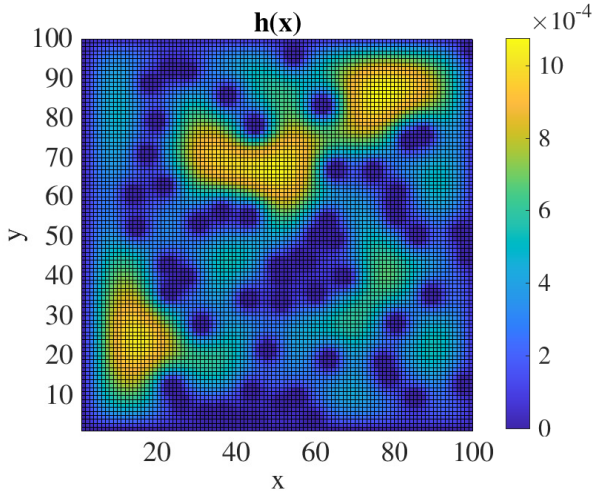

Fig. 1. Control barrier function for scenario I.

Standard RRT⋆ tends to exploit narrow gaps, often producing shorter but riskier paths (Fig. 2). This behavior is reflected in the low average safety value ($\bar{h} = 0.00057$). In contrast, Safe-CBF-RRT⋆ systematically rejects unsafe edges (48.7% of samples), forcing the tree to expand through regions of higher PSF values (Fig. 3). As a result, the mean path length increases from 118.57 to 147.86, while the mean safety metric improves by 16%.

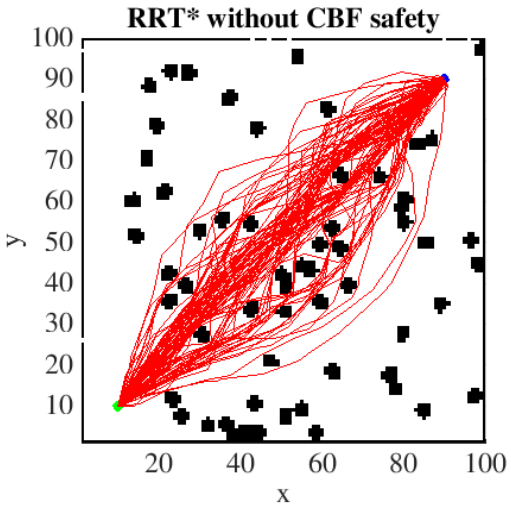

Fig. 2. Exemplary results of 100 different runs of standard RRT⋆ algorithm.

The higher number of iterations used by CBF-RRT⋆ is consistent with the stronger filtering mechanism, which prioritizes safety over direct progress toward the goal. Overall, Scenario I demonstrates that the proposed method effectively increases safety margins even in relatively easy environments.

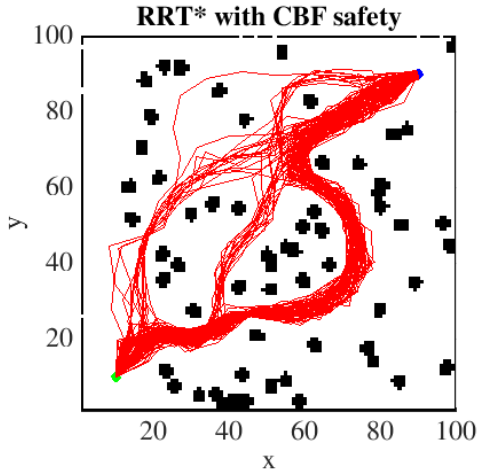

Fig. 3. Exemplary results of 100 different runs of CBF-RRT⋆ algorithm.

TABLE I
PERFORMANCE COMPARISON IN SCENARIO I.

Algorithm	Length	Average $h(x)$	Iterations	Rejections ratio
RRT⋆	118.57	0.00057	333	13.6%
CBF-RRT⋆	147.86	0.00066	563	48.7%

2) Scenario II: Scenario II introduces significantly tighter passages and higher obstacle density. The PSF field (Fig. 4) shows steeper gradients and narrow channels of admissible motion, making safety enforcement more restrictive.

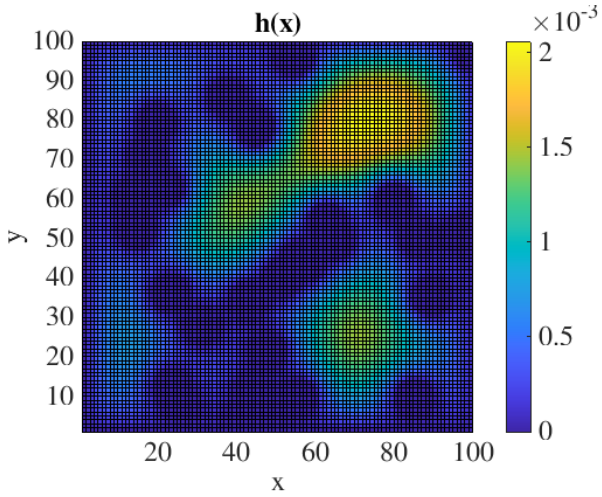

Fig. 4. Control barrier function for scenario II.

Standard RRT⋆ frequently selects unsafe extensions in this environment, reflected by a much higher rejection ratio (29.0%) compared to Scenario I. Its solutions remain relatively short (119.84), but operate close to obstacles, maintaining only a moderate safety average ($\bar{h} = 0.00085$).

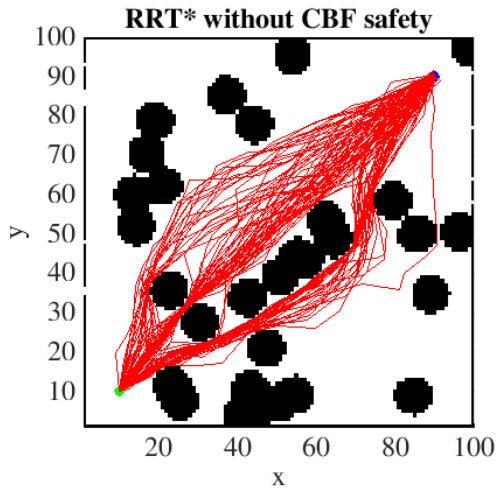

Fig. 5. Exemplary 100 different runs of standard RRT⋆ algorithm.

Safe-CBF-RRT⋆ must navigate through a more constrained safety landscape, leading to an even larger rejection ratio (59.9%) and correspondingly more conservative motion. Although the resulting paths are longer (130.95), they traverse regions with noticeably higher PSF values, improving the average safety metric by 29% relative to standard RRT⋆.

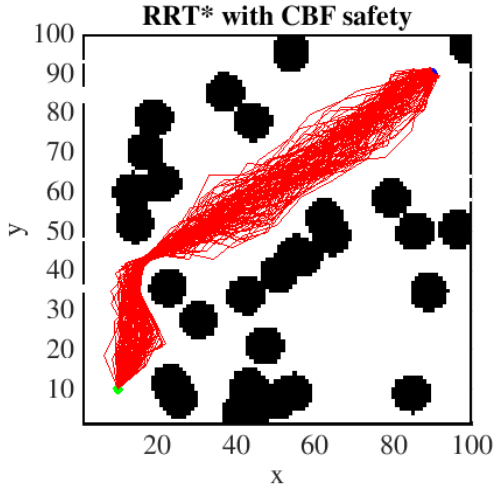

Fig. 6. Exemplary 100 different runs of CBF-RRT* algorithm.

Scenario II highlights the ability of the method to maintain safe operation under geometric constraints that naturally encourage risk-taking behavior in classical sampling-based planners.

TABLE II
PERFORMANCE COMPARISON IN SCENARIO II.

Algorithm	Length	Average $h(x)$	Iterations	Rejections ratio
RRT*	119.84	0.00085	345	29.0%
CBF-RRT*	130.95	0.00110	537	59.9%

3) Scenario III: Scenario III consists primarily of an open environment with a few concentrated high-risk areas. The PSF distribution (Fig. 7) features broad low-gradient regions interspersed with sharp drops near obstacles.

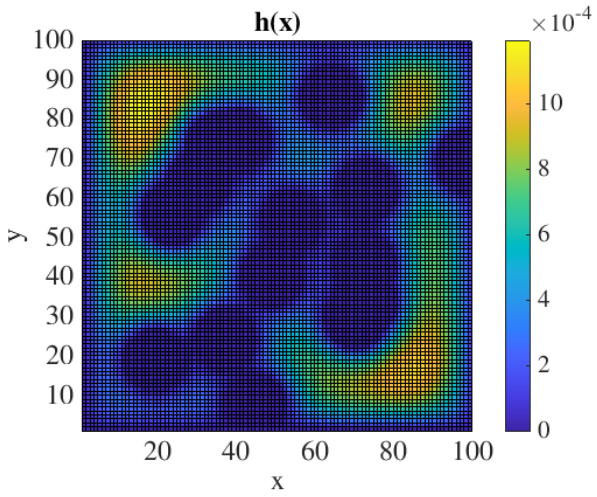

Fig. 7. Exemplary 100 different runs of standard RRT* algorithm.

In this configuration, standard RRT* is able to find relatively direct paths (122.71), but due to the scattered obstacle layout, it often crosses regions of low safety potential. This is reflected in the lowest average safety value among all scenarios ($\bar{h} = 0.00037$).

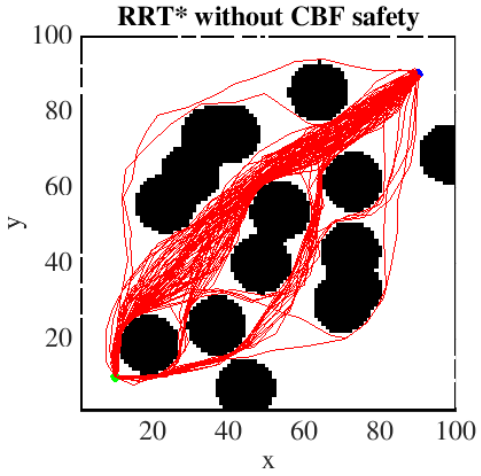

Fig. 8. Exemplary 100 different runs of standard RRT* algorithm.

Safe-CBF-RRT* improves the safety metric to $\bar{h} = 0.00048$ by rejecting 58.4% of unsafe expansions and steering the tree around low-PSF zones (Fig. 9). Interestingly, the number of iterations is slightly lower than for standard RRT*, indicating that large open areas reduce the need for rewiring and costly exploration. Despite the safety constraints, the method only marginally increases the path length (from 122.71 to 126.32), demonstrating that when safe alternatives are abundant, the penalty for enforcing CBF constraints becomes minimal.

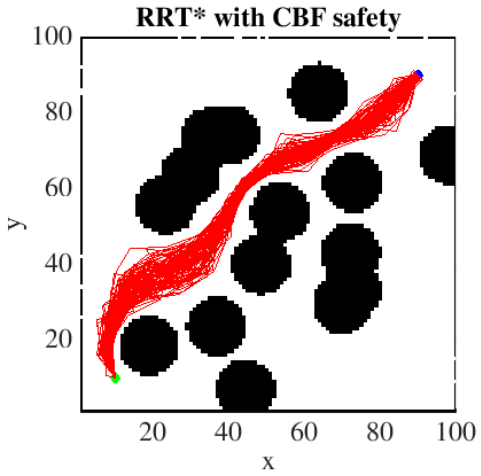

Fig. 9. Exemplary 100 different runs of CBF-RRT* algorithm.

Scenario III shows that the proposed planner adapts gracefully to environments where safe path options are naturally available, avoiding unnecessary conservatism while still improving safety.

TABLE III

PERFORMANCE COMPARISON IN SCENARIO III.

Algorithm	Length	Average $h(x)$	Iterations	Rejections ratio
RRT*	122.71	0.00037	421	39.8%
CBF-RRT*	126.32	0.00048	389	58.4%

C. Discussion

The CBF constraint effectively rejected paths approaching obstacles too closely. The Poisson field provided well-conditioned gradients, improving planner stability. Despite the additional checks, runtime increased only marginally due to efficient gridded interpolation.

VI. CONCLUSION AND FUTURE WORK

We presented *Safe-CBF-RRT**, a motion planning framework combining RRT* with a Control Barrier Function derived from a Poisson Safety Field. The approach unifies PDE-based potential field generation with formal safety guarantees **in a manner computationally lighter than tracking-based methods (e.g., FaSTrack)**, while actively shaping the path for safety. Future work will extend this method to kinodynamic models, uncertain environments, and real robotic experiments.

REFERENCES

[1] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431, 2019.

[2] G. Bahati and A. D. Ames. Safe set synthesis with tunable boundary gradients via poisson safety functions.

[3] G. Bahati, R. M. Bena, and A. D. Ames. Dynamic safety in complex environments: Synthesizing safety filters with poisson's equation, 2025.

[4] M. Chen, S. L. Herbert, H. Hu, Y. Pu, J. F. Fisac, S. Bansal, S. Han, and C. J. Tomlin. Fastrack:a modular framework for real-time motion planning and guaranteed safe tracking. *IEEE Transactions on Automatic Control*, 66(12):5861–5876, Dec. 2021.

[5] S. Karaman and E. Frazzoli. Incremental sampling-based algorithms for optimal motion planning, 2010.

[6] S. Karaman and E. Frazzoli. Sampling-based algorithms for optimal motion planning, 2011.

[7] S. M. LaValle. Rapidly-exploring random trees : a new tool for path planning. *The annual research report*, 1998.

[8] G. Yang, M. Cai, A. Ahmad, A. Prorok, R. Tron, and C. Belta. Lqr-cbf-rrt*: Safe and optimal motion planning, 2023.

[9] G. Yang, B. Vang, Z. Serlin, C. Belta, and R. Tron. Sampling-based motion planning via control barrier functions. In *Proceedings of the 2019 3rd International Conference on Automation, Control and Robots*, ICACR 2019, page 22–29. ACM, Oct. 2019.